

Оглавление

Введение.....	3
Титульный лист.....	3
Содержание.....	3
Нумерация страниц.....	4
Заголовки.....	5
Вставка оглавления.....	5
Итоги.....	6
Задание.....	6
Постановка задачи.....	7
Дано.....	7
Результат.....	8
При.....	9
Связь.....	10
Итог.....	10
Метод решения.....	11
Простая последовательность действий.....	12
Циклическая конструкция.....	12
Условная конструкция.....	13

Итог	14
Внешняя спецификация	14
Обычный вывод на экран	15
Вывод по условию	15
Повторный вывод.....	16
Итог	17
Алгоритм	18
Основы псевдокода.....	18
Ввод и вывод	19
Условная структура.....	20
Циклическая структура.....	21
Итог	22
Листинг.....	23
Отладка	Ошибка! Закладка не определена.
Тесты	23
Итог	23
Титры.....	24

Введение

Дорогие первокурсники!

Перед вами подробная инструкция по оформлению отчёта по лабораторным работам по курсу «Алгоритмизация и программирование».

При оформлении образца отчета используется Microsoft Office Word 2007. Все примеры и скриншоты сделаны для этой версии. У данного документа альбомная ориентация для того, чтобы все рисунки были хорошо видны. При оформлении отчёта используйте книжную ориентацию документа.

Важно то, что отчёт может быть один для двух или более программ из одной лабораторной работы. При этом внутри обязательных пунктов отчёта (таких как алгоритм, листинг, внешняя спецификация и прочее) необходимо сделать подзаголовки. В данном документе показан пример именно такого способа оформления (но ничто не мешает сделать два отдельных отчёта).

Титульный лист

Для создания титульного листа проще всего скопировать его из презентации по оформлению отчета по первой лабораторной работе. Мы не будем его сюда вставлять из-за частого изменения требований к этому листу. Просьба сделать столько столбцов для оценок, сколько программ в данной лабораторной работе. Обратите внимание, именно **программ**, а не заданий! Примечания о том, что задания должны быть выполнены в отдельных программах, в методичке есть, поэтому тут сложно запутаться.

Содержание

Скачайте презентацию к первой лабораторной и посмотрите на второй слайд под названием «Требования к оформлению отчёта». Что там стоит вторым пунктом? Правильно! Содержание! И не говорите, что вы этого не видели. К этому пункту относится и то, что все страницы вашего отчёта должны иметь нумерацию.

Как же это сделать? Всё очень просто! Начнём с нумерации страниц (см. рис.1).

Нумерация страниц

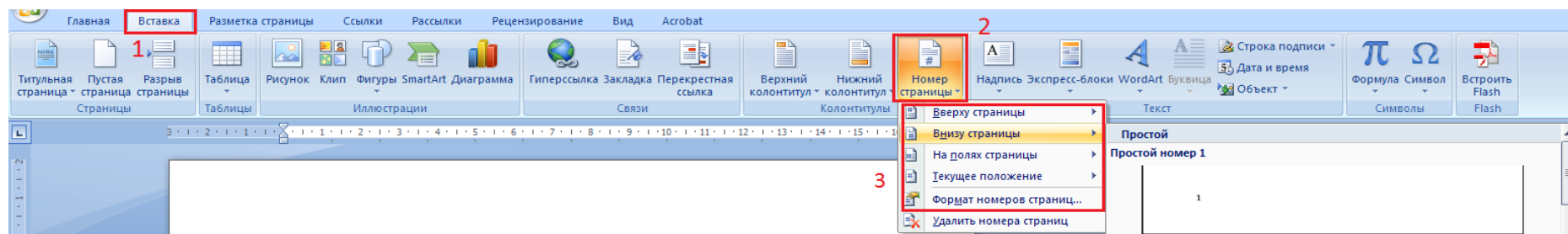


Рис. 1. Вставка номера страницы

Вам нужно выполнить следующие действия.

1. Открыть вкладку «Вставка».
2. Найти кнопку «Номер страницы» и нажать на неё.
3. Просмотреть появившееся меню. Уверяю, вы сможете найти множество интересных вариантов оформления, что-то да придётся вам по вкусу!

Отдельно отметим пункт «Формат номеров страниц». Лучше всего будет, если мы не будем нумеровать титульный лист. Поэтому в формате номеров нужно поставить начало нумерации с нуля (см. рис. 2).

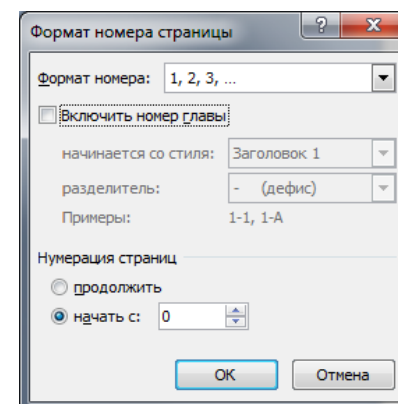


Рис. 2. Формат номера страницы

После всего этого Word перейдет в конструктор колонтитулов. В верхнем меню Конструктора вы можете поставить галочку «Особый колонтитул для первой страницы». При этом ноль из титульного листа будет скрыт (см. рис. 3). Вернуться в текст можно с помощью двойного щелчка мышью по нему.

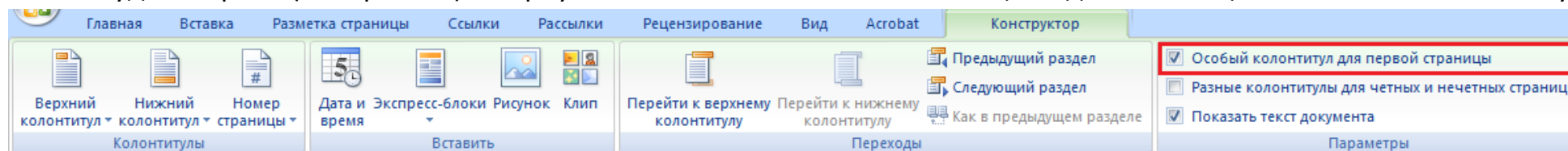


Рис. 3. Особый колонтитул для первой страницы

Со страницами мы разобрались. Перейдём к заголовкам.

Заголовки

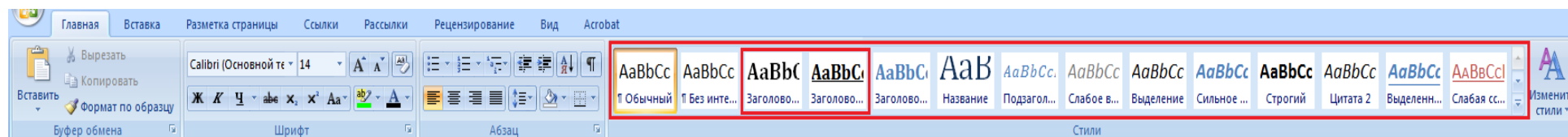


Рис. 4. Стили текста

На вкладке «Главная» вы найдёте таблицу стилей. Там собраны стили для обычного текста, для заголовков, названий и даже для цитирования (см. рис. 4). Всё это можно подстраивать, нажав на нужный стиль правой кнопкой мыши и выбрав пункт «Изменить» в контекстном меню. Для создания оглавления нам понадобятся заголовки нескольких уровней, а именно первого уровня, второго и, может быть, третьего. Подстройте их для себя.

Очень полезно пользоваться стилями. Особенно если вы решите в будущем оформить реферат, используя копирование и вставку фрагментов текста. Выделив текст с различными начертаниями и размерами шрифта, можно нажать на кнопку «Обычный» в стилях, чтобы весь этот фрагмент был оформлен единообразно.

Вставка оглавления

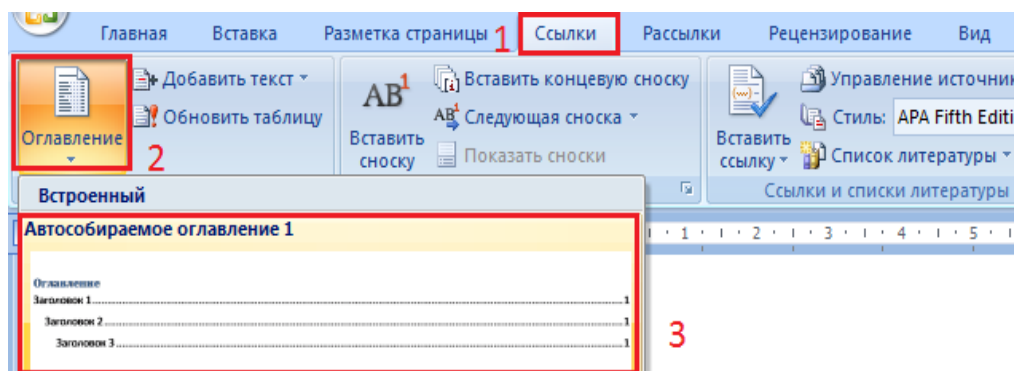


Рис. 5. Вставка оглавления

Содержание в Word называется «Оглавление». Но это не критично, всё-таки смысл одинаковый. Для создания оглавления надо сделать следующее.

1. Перейти во вкладку «Ссылки».
2. Найти кнопку «Оглавление» и щелкнуть по ней мышью.
3. Выбрать из появившегося меню то, что вам больше всего понравилось!

Итоги

Что же должно было получиться? На данном этапе у нас есть такое оглавление/содержание.

Оглавление

Введение	1
Титульный лист.....	1
Содержание	1
Нумерация страниц	1
Заголовки.....	3
Вставка оглавления.....	3
Итоги.....	4

Если вы щёлкните на любое место оглавления, то перейдёте в него. Оглавление - это отдельная структура. В верхнем правом углу оглавления есть кнопка «Обновить таблицу».

Задание

Очень простой пункт. Вам необходимо верно скопировать из методички (Ctrl+C) и вставить в отчет (Ctrl+V) текст ваших заданий. Надо их разделить на пункты, чтобы не было сплошного текста. Если необходимо сделать задания в разных программах, то лучше всего это указать в конце данного пункта.

Пример:

1. Из данного массив $R[1:n]$ удалить все положительные элементы, расположенные между элементом с заданным индексом и первым максимальным элементом, включая сами эти элементы.
 2. Дан массив целых положительных чисел $X[1:n]$. Вычислить произведение сумм цифр каждого элемента массива.
- Задания 1 и 2 выполняются в разных программах.

Данные задания мы используем для следующего пункта.

Постановка задачи

Над этим пунктом придется подумать (и это очень полезно). Постановка задачи нужна для того, чтобы студенты научились анализировать задание, добывать из него необходимую информацию и осмысливать её. На этапе постановки необходимо прикинуть, а что же придётся писать в программе. Лучше всего начать именно с этого пункта, а не с самой программы, как это иногда делают некоторые студенты. И дело даже не в том, что потом на защите можно будет пафосно заявить, что вы начали не с программы, как все остальные, а в том, что это и вправду поможет избежать многих недочётов.

Постановка задачи содержит четыре пункта: «Дано», «Результат», «При», «Связь». Названия всех пунктов необходимо подчеркнуть. Рассмотрим каждый из них в отдельности.

Дано

В пункте «Дано» необходимо указать исходные данные. Если заданий несколько, то их надо также разбить по пунктам. Иногда в задаче словами сказано, что нам что-то задано пользователем, а обозначения переменной нет. В таком случае название переменной даём сами. Если в задаче есть слова «дан» или «данный массив», то это значит, что пользователь вводит элементы с клавиатуры. И никак иначе. Это связано с тем, что в дальнейшем при тестировании программы нам потребуются массивы, имеющие некоторые особенности (например, не содержащие ни одного нуля, состоящие только из отрицательных элементов и т.п.)

Пример:

Дано:

1. $n, R[1:n], k_z$, где k_z – заданный индекс;
2. $n, A[1:n]$.

Необходимо отметить, что при обработке массива необходимо вначале ввести длину, а потом сам массив. Это не безразлично для нас, поскольку тоже помогает воспитать в вас правильное отношение к алгоритмизации задачи. Если для вас всё равно, что поставить первым, то это говорит лишь о том, что вам не ясно, зачем мы вообще пишем постановку задачи.

Результат

В отличие от пункта «Дано», в «Результате» мы описываем то, что у нас должно быть на выходе из программы (отдельно для каждого задания). Результат должен соответствовать тому, что требуется найти по условию задачи.

Внимание! В данном пункте необходимо рассмотреть все особые случаи, которые могут возникнуть при решении задачи. Примеры: «Полное удаление массива», «Полное обнуление» и т.п. Это важно, поскольку вы должны предусмотреть эти случаи заранее, а не на этапе программирования или алгоритмизации.

В будущем вы познакомитесь с объектно-ориентированным программированием, где необходимо будет передавать массивы или матрицы между объектами. При этом в объекте-получателе работа зачастую происходит с копией матрицы или массива, а не с оригиналом. Если матрица будет содержать размеры 10000x10000, то мало того, что её сложно обработать, так ещё и надо будет передать что-то обратно на перезапись. И куда легче передать не матрицу 10000x10000, а фразу «После обработки матрицы больше нет». Ещё интереснее ситуация с обнулением, ведь изначально можно ввести массив или матрицу, состоящую полностью из нулей. И получается, что полностью обнулённая матрица и матрица, которая изначально состоит из нулей, - две разные вещи.

Пример:

Результат:

1. $R[1:k]$, где k – длина массива после удаления, или сообщение «Полное удаление массива», или сообщение «Удалений не было»;
2. Pr , где Pr – произведение сумм цифр каждого элемента массива.

Заметим, что в «Результате» и последующих аналитических пунктах не стоит писать нечто вроде «Размер матрицы меньше 1, невозможно выполнить задание». Ограничения на допустимость вводимых данных мы записываем в следующем пункте постановки.

При

Пункт «При» – это ограничения на **вводимые** данные. Внимание! Повторим ещё раз! Вводимые. Так что ограничения нам нужны только на те данные, что мы перечислили в пункте «Дано». Будем считать, что пункт «При» предназначен для пользователя, чтобы он не вводил того, что не допустимо. Правда это не освобождает вас от «защиты от дурака» (foolproof) в программе, поскольку пользователи программы бывают разные.

Пример:

При:

1. $n \in \mathbb{N}$, $n \leq l_{\max}$, $kz \in \mathbb{N}$, $kz \leq n$;
2. $n \in \mathbb{N}$, $n \leq l_{\max}$, $A[1:n] \in \mathbb{N}$.

Если припомнить лекции по математическому анализу, то вы должны знать, что натуральные числа ($\mathbb{N}$) не содержат в себе 0 и отрицательных, а ещё дробных чисел. Поэтому писать, что $n > 0$ не нужно. Для числа k , которое у нас возникает в пункте «Результат», мы ограничения не пишем, поскольку это число нам не дано. Это значение вычисляется в самой программе.

Связь

Самый трудный пункт для первокурсников (да и для второкурсников). Поэтому все самые важные объяснения вы получите или уже получили на лекциях и семинарах, а здесь вы найдете лишь несколько примечаний.

Пункт «Связь» – это математическое описание того, что вы должны найти или сделать. К примеру, если сказано поменять местами минимальный и максимальный элементы, то в связи надо написать:

1. связь для поиска минимума;
2. связь для поиска максимума;
3. перестановку.

Итого 3 строки. Не забывайте в них использовать знаки Σ (сумма), $\forall$ (для каждого), $\exists$ (существует), $:$ (такой, что).

Последнее. Если вам дана формула, то, как правило, в этом пункте можно написать «см. в условии».

Итог

Итак, соберём всё, что у нас получилось в единую постановку.

Дано:

1. $n, R[1:n]$, kz , где kz – заданный индекс;
2. $n, A[1:n]$.

Результат:

1. $R[1:k]$, где k – длина массива после удаления, или сообщение «Полное удаление массива», или сообщение «Удалений не было»;
2. Pr , где Pr – произведение сумм цифр каждого элемента массива.

При:

1. $n \in \mathbb{N}, n \leq l_{\max}, kz \in \mathbb{N}, kz \leq n$;

2. $n \in \mathbb{N}$, $n \leq l_{\max}$, $A[1:n] \in \mathbb{N}$.

Связь:

1. $\exists n_{\max}: n_{\max} \in [1:n]: \forall i=1, \overline{n} R[i] \leq R[n_{\max}]. \nexists t=1, \overline{n_{\max}-1}: R[t] > R[n_{\max}]$

при $n_{\max} > k_z$ $c = n_{\max}$, $n_{\max} = k_z$, $k_z = c$

$\forall i = \overline{n_{\max}, n}: R[i] \leq 0$ и $i \leq k_z$ или $i > n_p \exists t: R[t] = R[i]$, $t = n_{\max}, k$

2. $\forall i = \overline{1, n} A[i] = \overline{a_{ik_i-1} a_{ik_i-2} \dots a_{i1} a_{i0}}$, где k_i – количество цифр в $A[i]$

$$pr = \prod_{i=1}^n \sum_{j=0}^{k_{i-1}} a_{i,j}$$

Перед тем, как перейти к следующему пункту отчёта, заметим, что в пункте «Связь» мы не используем знак присваивания ($:=$), а применяем математический символ, то есть обычное равенство ($=$).

Метод решения

Метод решения показывает нам математический способ нахождения результата, он еще ближе к алгоритму. Важно, что мы тоже используем математический язык (это про «равно», а также символы « $\geq$ » и « $\leq$ »). Здесь мы описываем пошагово как будем искать то, что записали в связи. Нам **не нужно** описывать крайние случаи и выводы. То есть нет проверки типа «если мы не нашли максимум, то это выполняться не будет». Делаем всё так, будто обязательно всё найдётся и сделается. В конце концов, это не алгоритм, а лишь описание блоков поиска. Для собственного спокойствия можно представить, что они не объединены между собой. Для поиска результата вам нужны будут условия и циклы. Для них в методе решения задачи есть свои правила оформления. Именно с них мы и начнём.

Для описания правил оформления метода выберем более простой пример. Он не имеет ничего общего с заданиями, которые мы рассматривали выше.

Простая последовательность действий

Что такое простая последовательность действий вы изучали на первой или второй лекции – это последовательность действий, выполняемых одно за другим. И поэтому мы пишем их в столбик. Для каждого действия своя строка. Можно написать в строчку несколько действий, разделив их точкой с запятой (;). После действия, если оно последнее в строке, ставить точку с запятой не нужно, это ошибка. Исключения из правила написания такого алгоритма лишь одно, но о нём поговорим чуть позже.

Примеры:

nmax = 0

nmax = 0; max = 1e308

max = 1e308

Циклическая конструкция

Существует 3 вида циклов. Начнём с самого простого – цикла со счётчиком. Это *for i:=1 to n do* в синтаксисе Паскаля. Как этот цикл оформляется в методе? Первой строчкой пишем описание самого цикла: для $i=\overline{1,n}$. Черта сверху важна, поскольку показывает нам, что мы просматриваем все номера элементов массива от 1 до n, а не только первый и последний номер. Дальше в столбик записываются все действия, которые мы выполняем в цикле. А для обозначения того, какие именно действия входят в цикл, мы с левой стороны ставим фигурную скобку, которая охватывает сам цикл и всё, что в него входит.

Пример:

$$\left\{ \begin{array}{l} \text{для } i=\overline{1,n} \\ k=k+1 \\ A[k]=B[i] \\ n=k \end{array} \right.$$

В примере, приведённом выше, мы два действия выполняем в цикле, а последнее – после него. Действия могут выполняться в двух (вложенных) циклах. В этом случае используем две фигурные скобки. Если во вложенных циклах только одно действие, то можно их записать в общей строке, и использовать одну скобку.

Пример с одним действием в теле вложенного цикла:

$$\left\{ \begin{array}{l} \text{для } i=\overline{1, n}, j=\overline{1, m}, \\ A[i,j]=0 \end{array} \right.$$

Рассмотрим пример с несколькими скобками для обозначения вложенных циклов.

$$\left\{ \begin{array}{l} \text{для } i=\overline{1, n} \\ k=k+1 \\ \left\{ \begin{array}{l} \text{для } j=\overline{1, m} \\ X[k]=A[i,j] \\ k=k+2 \end{array} \right. \end{array} \right.$$

Циклы с предусловием и постусловием (while и repeat-until) оформляются похожим образом. Приведем примеры:

$\begin{array}{l} i=0 \\ \left\{ \begin{array}{l} \text{Пока } i < 5 \\ i=i+1 \end{array} \right. \end{array}$	$\begin{array}{l} i=0 \\ \left\{ \begin{array}{l} i=i+1 \\ \text{До } i=5 \end{array} \right. \end{array}$
--	--

Конечно, если в цикл с пред- или постусловием вложен еще один цикл (со счетчиком или пред- или постусловием), то необходимо использовать несколько фигурных скобок.

Условная конструкция

Как оформлять условную конструкцию? В этом случае, мы пренебрегаем правилами оформления линейного алгоритма. Вначале мы пишем в строчку, через запятую, действия, которые выполняются по условию, а потом само условие. При

большом количестве действий можно заключить их в круглые скобки, но всё также записать в строчку. Если есть ветвь «иначе», то оформляем ее так же, только в конце пишем «в противном случае».

Пример:

k=n; i=0; j=0

{ пока k>0
{ (i=i+1, C[i]=A[k], k=k-1), если A[k]>5
{ (j=j+1, R[j]=A[k], k=k-1), в противном случае.

В приведенном выше методе можно кое-что упростить. Если вы заметили что именно, то это большой успех.

Если нужно использовать несколько условий, то ставим между ними связку «и», либо «или», в зависимости от ситуации.

Итог

Осталось лишь всё полученное выше соединить воедино. Напишем метод решения для задачи со следующим условием: «Обнулить все элементы массива A[1:n], расположенные после последнего отрицательного элемента».

no=0

{ для i=1, n
{ no=i, если A[i]<0
{ для i=no + 1, n
{ A[i]=0

До начала цикла (это важно!) мы придаём начальное значение переменной no, потом вычисляем индекс последнего отрицательного элемента. На последнем этапе обнуляем все элементы, расположенные после найденного. Никаких проверок на то, что отрицательное есть, что оно последнее делать не нужно. А ещё в методе нет ввода данных и вывода результатов.

Внешняя спецификация

Можно было бы назвать этот пункт самым творческим. Здесь много графических объектов, а ещё всё должно быть оформлено не только правильно, но и красиво. Во внешней спецификации мы показываем, что программа выводит на

экран, и как именно вводятся данные и выводится результат. Тут надо понимать, что некоторые фрагменты сообщений могут выводиться несколько раз, какие-то части при определенных условиях мы можем не выводить на экран. Для всего этого тоже есть свои правила оформления, которые мы сейчас рассмотрим.

Обычный вывод на экран

Это то, что появляется на экране ровно один раз. Необходимо заключить весь текст в прямоугольник, который символизирует собой экран. Важно заметить, что если вы выводите значения каких-либо переменных, то необходимо заключить их в двойные угловые «скобки» (например: <<A[1]>>). В одинарные «скобки» мы заключим переменную в том случае, если требуется ввести её значение. Например, выведем приветствие пользователю, попросим ввести значение переменной, а затем выведем это значение на экран. Заметим, что при создании прямоугольника изначально он будет белым, а потому перекроет весь текст, который ранее был написан. Поэтому нужно в контекстном меню выбрать пункт «Формат автофигуры» и изменить там прозрачность элемента.

Привет!

Введите число: <A>

Вы ввели число <<A>>.

Вывод по условию

А что делать, если у нас несколько вариантов вывода? Ведь часто в результате мы можем получить только одно из нескольких сообщений. В таком случае необходимо перед прямоугольником написать, при каком условии это сообщение будет выводиться. То есть перед первым окном пишем «При <условие>», а второе окно, если оно есть, оформляем с помощью «Иначе». При этом справа от прямоугольников, мы ставим фигурные скобки (не включая в них сами условия). Для каждого прямоугольника отдельная скобка. Если после «Иначе» есть ещё вариация, то она вся заключается в фигурную скобку (включая внутренние условия). Рассмотрим два варианта выполнения одного и того же вывода в зависимости от выполнения условия.

При $k = 0$

Полное удаление массива!

Иначе

При $k = n$

Не было удалений!

Иначе

<<A[1]>> <<A[2]>> ... <<A[k]>>

При $k = 0$

Полное удаление массива!

При $k = n$

Не было удалений!

Иначе

<<A[1]>> <<A[2]>> ... <<A[k]>>

Это два варианта одного и того же вывода. Какой же выбрать? Тот, который позже будет соответствовать вашему алгоритму. Все части отчёта связаны и должны соответствовать друг другу. Наверное, именно из-за этого студенты часто пытаются писать в начале программу, а потом отчёт. Обратите внимание на вывод массива. Правильно выводить массив именно так (вариант вывода <<A[1:k]>> неверен). Так же верный вариант приведен ниже (подробнее о повторном выводе рассказывается в следующем разделе):

$\left. \begin{array}{l} i=\overline{1, k} \\ \end{array} \right\}$

<<A[i]>>

$\left. \begin{array}{l} \end{array} \right\} \star$

Повторный вывод

Обычный цикл со счётчиком во внешней спецификации отсутствует. Как уже показано выше, каждый элемент придётся выводить отдельно. А вот с циклами с пред- и постусловием всё несколько иначе. Во всех лабораторных работах будет хотя бы одно окно, которое может появиться несколько раз. Это проверка правильности ввода данных. Здесь необходимо написать в окне, что именно выводится или запрашивается на экране. После прямоугольника следует записать «До <условие выхода из цикла>». Конечно, можно было бы привести пример для цикла «Пока», но цикл «до» в данном случае

лаконичнее и логичнее. Фигурные скобки ставятся с двух сторон, а у правой скобки сверху ставим ещё и звёздочку. Выглядит всё это так:

{ Введите n: <n> } ★

До $n > 0$ и $n \leq \text{Imax}$

Итог

Как обычно, попробуем собрать всё воедино!

Лабораторную работу №х

Придумал(а): Ерохина Елена Альфредовна

{ Введите длину массива n: <n> } ★

До $n > 0$ и $n \leq \text{Imax}$

Введите массив:

<A[1]> <A[2]> ... <A[n]>

При $k = 0$

{ Полное удаление массива! }

Иначе

При $k = n$

{ Не было удалений! }

Иначе

{ <<A[1]>> <<A[2]>> ... <<A[k]>> }

Внешняя спецификация готова.

Алгоритм

Итак, подходим к одному из самых важных пунктов – алгоритму. Чем алгоритм отличается от программы? Разберём это на примере важной для всех студентов вещи – пельменей. Алгоритм – это инструкция или словесное описание того, как готовить пельмени. А программа – это уже непосредственно сам процесс готовки пельменей. Второе более подробно, и при этом можно увидеть результат.

Алгоритм в данном курсе требуется записывать на псевдокоде, который имеет свои стандарты. С ними и будем сейчас знакомиться. Для начала стоит сказать, что в псевдокоде тоже используются математические знаки. В псевдокоде нет функций, характерных для языков программирования (например, таких как `mod`, `div` Паскаля), всё это мы заменяем аналогами из математики. Для кратности зачастую это `:`, а для `div` – `\`. Все ключевые слова в структурах на псевдокоде необходимо подчёркивать. Алгоритм также может включать в себя описание функций и процедур.

Основы псевдокода

Любой алгоритм, для программы, процедуры или функции, начинается со слова «Алг» и названия алгоритма. Если эти слова вам не знакомы, то не печальтесь, всё ещё впереди. Если это функция, то вместо названия пишем «Функция <имя функции>». Далее для функции пишем слово «Вход:», после чего перечисляем имена всех входных переменных. В процедуре (а иногда и в функции) вдобавок пишем ещё «Выход:» и записываем имена всех выходных переменных. Если входных, либо выходных данных нет, то соответствующий пункт можно пропустить. Больше разницы между алгоритмом программы и функцией не будет.

Далее пишем слово «Нач», нажимаем `Enter` и `Tab`. В псевдокоде необходимо делать отступы, чтобы была видна структура алгоритма. Если этого не делать, то для анализа алгоритма потребуется на порядок больше времени. Об отступах всё будет рассказано в пункте «Листинг», можете прямо сейчас посмотреть туда. Алгоритм заканчивается словом «Кон». В псевдокоде используется оператор присваивания (`:=`), а вот точку с запятой можно использовать только для разделения операторов, расположенных в одной строке. Рассмотрим пример оформления алгоритма для процедуры.

Алг «Процедура summa»

Вход: n, A[1:n]

Выход: sum

Нач

{операторы для выполнения}

Sum:=0

цикл от i:=1 до n

Sum:=Sum+A[i]

кц

Кон

Если удалить строки «Вход» и «Выход», то будет обычный алгоритм. Можно считать, что эти строки в данном примере заменяют ввод данных и вывод результата.

Ввод и вывод

Ввод и вывод – чуть ли не единственное, что не нужно подчёркивать в псевдокоде. (Напомним, что подчеркиваются ключевые слова в структурах, а ввод, вывод и присваивание – это операции.) Ввод и вывод выглядят так:

ввод (n, A[1: n])

вывод («Нет удаления»)

вывод(A[1: k])

Заметим, что поскольку алгоритм – это лишь инструкция, то и вводить значения переменных можно без проверки допустимости данных. Проверять исходные данные на допустимость нужно будет в программе. Вводить каждый элемент массива отдельно тоже не стоит (для экономии времени). Поэтому лучше всего использовать вариант ввода массива из первой строки примера. Так вы не сделаете ошибок, по крайней мере, грубых (кто-то забывает ставить скобки, вместо

диапазона изменения номеров ставит текущий номер, и т.д., и т.п.). Хотя если весь ввод расписать подробно, то ошибки не будет.

Условная структура

Переходим к алгоритмическим структурам (они же конструкции). Отметим, что в условиях будут подчёркиваться логические связки «и», «или», «не». Условная конструкция начинается со слова «если», затем идёт условие, а после него – «то». Все ключевые слова в конструкции подчёркиваем. Дальше идёт тело условной структуры. Хочется заметить, что в псевдокоде нет замены оператору «begin», зато условная конструкция всегда завершается словом «всё». Пожалуйста, не забывайте о букве ё! Хороший пример того, насколько она важна:

Мы все здесь передохнём.

Мы все здесь передохнем.

После этого лирического отступления рассмотрим пример условной конструкции.

если <условие 1> и (<условие 2> или <условие 3>) то

<операторы, выполняемые при истинном условии>

[иначе

<операторы, выполняемые при ложном условии>]

всё

Часть, оператора, заключенная в квадратные скобки, может отсутствовать. «Иначе» не стоит рассматривать отдельно от начального условия, ведь оно лишь продолжение условной конструкции. Поэтому отдельное «всё» для ветви «иначе» мы не ставим. Заметим, что даже при выполнении внутри условия только одного оператора, необходимо в конце конструкции поставить «всё».

Циклическая структура

В псевдокоде имеется всего три вида циклов. Начнем с циклов с пред- и постусловием. Цикл с предусловием начинается со слов «цикл-пока», затем записывается условие продолжения цикла. С новой строки (и с отступом) – операторы. А завершает всю конструкцию слово «кц» - конец цикла. Важно то, что, как и в случае с условной конструкцией, «кц» обязательно записывать в конце каждого цикла. Независимо от того, сколько операторов выполняется в теле цикла.

Для цикла с постусловием всё очень похоже, хотя там и есть «мелочь», про которую обычно забывают. Начинаем со слова «цикл», а потом с новой строки, после отступа, пишем все действия внутри цикла. Затем пишем «до <условия выхода из цикла>», а завершаем конструкцию новой строкой с сокращением «кц».

Пример:

<u>цикл</u>	<u>цикл-пока</u> <условие продолжения цикла>
<действия внутри цикла>	<действия внутри цикла>
<u>до</u> <условие завершения цикла>	<u>кц</u>
<u>кц</u>	

Рассмотрим цикл со счётчиком. Перейдём сразу к правилам его оформления.

цикл от <переменная цикла> := <начальное значение> до <конечное значение> [шаг <величина шага>]

 <действия в цикле>

кц

То, что внутри квадратных скобок, писать не обязательно, если шаг цикла равен 1. Отдельное внимание следует обратить на то, что в заголовке цикла используем оператор присваивания, а не обычное «равно». Все ключевые слова должны быть подчёркнуты. В конце обязательно записываем «кц» вне зависимости от того, сколько действий располагается в теле цикла.

Итог

Приведем пример алгоритма, записанного на псевдокоде. Посчитаем сумму элементов массива $A[1:n]$, которые больше -5 и меньше 5. Алгоритм простой, но в нем используется большая часть рассмотренных выше структур.

алг «Сложение чисел»

нач

ввод($n, A[1:n]$)

sum:=0; flag:=ложь

цикл от $i:=1$ до n

если $A[i] > -5$ и $A[i] < 5$ то

 sum:=sum+A[i]; flag:=истина

все

кц

если flag то

 вывод(sum)

иначе

 вывод(«Нет результата»)

все

кон

Обратите особое внимание на отступы и подчёркивания.

Напоследок заметим, что после распечатки отчёта вы можете карандашом для себя соединить все окончания структур с их началом. Это вполне допустимо и позволит избежать большинства ошибок.

Листинг

Пункт, в котором просто надо привести код программы. Это довольно просто, самое сложное – правильно расставить отступы в программе. Цель - научить вас составлять программы так, чтобы они были понятны и их можно было бы легко читать и модифицировать. Отступы мы делаем для каждой структуры так, чтобы можно было легко понять где её начало и конец.. Так гораздо удобнее понять структуру алгоритма в целом.

Тесты

Тесты представляют собой отдельную таблицу для каждой программы. Лучше всего отдельно подписать каждую таблицу при помощи заголовка второго уровня. В таблице должны быть приведены номера тестов, исходные данные и результаты. Тесты должны быть исчерпывающими. По сути это все возможные варианты ваших результатов, приведенные в постановке задачи. Пример, где размер массива или матрицы выходит за ограничения «При» не обязателен, но его отсутствие не считается ошибкой. **Пример** (для задания 1 из пункта «Задание»):

№	Исходные данные	Результат
1	$n = 4, A = \{-5, 4, 6, 5\}, kz = 1$	$A = \{-5, 6, 5\}$
2	$n=3, A = \{5, 4, 6\}, kz = 1$	Полное удаление массива
3	$n=1, A = \{-5\}, kz = 1$	Нет удаления

Вы вполне можете дописать ещё тесты, чтобы показать некоторые моменты в своей программе, но тесты не должны повторяться.

Итог

Надеюсь, что данный документ поможет избежать дальнейших ошибок в отчётах и ускорит их проверку. Желаем всем творческих успехов!

Титры

Составил: Лескин Пётр Андреевич.

Отредактировала: Ерохина Елена Альфредовна.

Одобрили: Ерохина Елена Альфредовна, Хрустова Диана Владимировна.

Особая благодарность от автора: Кравченко Алле, Быстровой Анне, Русанову Андрею.